

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.(BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 3

Duration : 1 Hour
Total Marks:30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Constraint-Based Problem Solving

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that:

Constraints:

- i. D1 cannot work Night shift
- ii. D2 must work before D3 (Morning < Afternoon < Night)
- iii. Only one doctor per shift
- iv. D3 cannot work Morning
- v. All doctors must be assigned exactly one shift

Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

Constraints:

- i. Movement allowed: Up, Down, Left, Right
- ii. Cost of each move = 1
- iii. Cannot pass through obstacles
- iv. Use Manhattan Distance heuristic

Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost.

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.

The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

Constraints:

- The robot can move up, down, left, right
- Some cells are blocked (obstacles) and cannot be traversed
- The robot has limited sensing capability (can only observe adjacent cells)
- Each movement has a cost of 1, but entering risky zones has an additional cost of 2
- The robot must minimize total cost while ensuring safe navigation
- The robot must update its knowledge dynamically (online search scenario)

Tasks:

- a) Analyze all the given constraints and explain how they affect the robot's decision-making process.

- b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.
- c) Demonstrate how the robot applies AI concepts such as:
- Intelligent agents
 - Rationality
 - Environment type
- d) Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Question 1: Constraint-Based Problem Solving using Backtracking Search

Problem Statement

A hospital must assign three doctors (**D1, D2, D3**) to three shifts (**Morning, Afternoon, Night**) while satisfying the given constraints.

Constraints

1. D1 cannot work Night shift.
 2. D2 must work before D3 (Morning < Afternoon < Night).
 3. Only one doctor can work in one shift.
 4. D3 cannot work Morning.
 5. Every doctor must be assigned exactly one shift.
-

1. CSP Formulation

A Constraint Satisfaction Problem (CSP) consists of:

- Variables
- Domains
- Constraints

Variables

- D1
- D2
- D3

Domains

Each doctor can initially work any shift.

Doctor	Domain
D1	{Morning, Afternoon, Night}
D2	{Morning, Afternoon, Night}
D3	{Morning, Afternoon, Night}

2. Constraint Checking

Apply constraints one by one.

Constraint 1

$D1 \neq \text{Night}$

Domain becomes

$D1 = \{\text{Morning, Afternoon}\}$

Constraint 2

$D3 \neq \text{Morning}$

Domain becomes

$D3 = \{\text{Afternoon, Night}\}$

Constraint 3

D2 must work before D3

Possible combinations

Morning $\rightarrow$ Afternoon ✓

Morning $\rightarrow$ Night ✓

Afternoon $\rightarrow$ Night ✓

Constraint 4

Only one doctor per shift.

No duplicate assignments.

Constraint 5

Every doctor gets exactly one shift.

3. Backtracking Search

Initial State

D1 = ?

D2 = ?

D3 = ?

Step 1

Assign

D1 = Morning

Constraint Check

✓ D1 is not Night.

Continue.

Step 2

Assign

D2 = Afternoon

Constraint Check

Remaining shift = Night.

Step 3

Assign

D3 = Night

Check Constraints

✓ D3 is not Morning

✓ D2 before D3

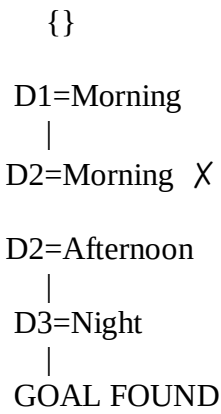
✓ One doctor per shift

✓ Every doctor assigned

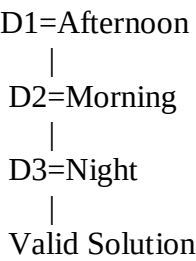
Solution Found.

Backtracking Tree

Start



Another Branch



Intermediate Assignments

Step	Assignment	Constraint Check
1	D1=Morning	Valid
2	D2=Afternoon	Valid
3	D3=Night	Valid
Goal Complete Assignment Success		

Final Valid Doctor Schedule

Shift	Doctor
Morning	D1
Afternoon	D2

Shift	Doctor
Night	D3

Result

Using **Backtracking Search**, all constraints are satisfied.

The valid assignment is

Morning → D1

Afternoon → D2

Night → D3

Code:

Backtracking Search for Doctor Shift Assignment

```
doctors = ["D1", "D2", "D3"]
```

```
shifts = ["Morning", "Afternoon", "Night"]
```

```
assignment = {}
```

Constraint checking

```
def is_valid(doctor, shift):
```

```
    # D1 cannot work Night
```

```
    if doctor == "D1" and shift == "Night":
```

```
        return False
```

```
    # D3 cannot work Morning
```

```
    if doctor == "D3" and shift == "Morning":
```

```
        return False
```

Only one doctor per shift

if shift in assignment.values():

return False

return True

def backtrack(index):

if index == len(doctors):

Check D2 before D3

order = {"Morning":1, "Afternoon":2, "Night":3}

if order[assignment["D2"]] < order[assignment["D3"]]:

print("\nValid Schedule Found")

print("-----")

for d in doctors:

print(d, "->", assignment[d])

return True

return False

doctor = doctors[index]

for shift in shifts:

print(f"Trying {doctor} -> {shift}")

```
if is_valid(doctor, shift):  
    assignment[doctor] = shift  
    print("Accepted")
```

```
if backtrack(index+1):  
    return True
```

```
print("Backtracking from", doctor)  
del assignment[doctor]  
else:  
    print("Rejected")
```

```
return False
```

```
backtrack(0)
```

output:

```

... backtrack(0)
...
Trying D1 -> Morning
Accepted
Trying D2 -> Morning
Rejected
Trying D2 -> Afternoon
Accepted
Trying D3 -> Morning
Rejected
Trying D3 -> Afternoon
Rejected
Trying D3 -> Night
Accepted

Valid Schedule Found
-----
D1 -> Morning
D2 -> Afternoon
D3 -> Night
True

```

Question 2 : Robot Navigation using BFS Search

Problem Statement

A robot moves in a 5×5 grid.

Movement allowed

- Up
- Down
- Left
- Right

Each move costs **1**.

Robot cannot cross obstacles.

5×5 Grid

```

S . . X .
. X . X .
. X . . .

```

. . X X .
. . . . G

Legend

S = Start

G = Goal

X = Obstacle

. = Free Cell

Coordinates

S = (0,0)

G = (4,4)

Obstacle Map

(0,3)

(1,1)

(1,3)

(2,1)

(3,2)

(3,3)

Initial State

(0,0)

Goal State

(4,4)

BFS Queue Updates

Step	Queue
------	-------

1	[(0,0)]
---	---------

2	[(1,0),(0,1)]
---	---------------

Step	Queue
3	[(0,1),(2,0)]
4	[(2,0),(0,2)]
5	[(0,2),(3,0)]
6	[(3,0),(1,2)]
7	[(1,2),(4,0),(3,1)]
8	[(4,0),(3,1),(2,2)]
9	Continue until Goal

Node Expansion Table

Expanded Node	New Nodes
(0,0)	(1,0),(0,1)
(1,0)	(2,0)
(0,1)	(0,2)
(2,0)	(3,0)
(3,0)	(4,0),(3,1)
(3,1)	(4,1)
(4,1)	(4,2)
(4,2)	(4,3)
(4,3)	(4,4)

Goal reached.

Manhattan Distance Heuristic

Formula

$$h(n) = |x - \text{goal_x}| + |y - \text{goal_y}|$$

Example

Node

(2,2)

Goal

(4,4)

$$|2-4| + |2-4|$$

$$= 2 + 2$$

=4

Optimal Path

(0,0)

↓

(1,0)

↓

(2,0)

↓

(3,0)

↓

(3,1)

↓

(4,1)

↓

(4,2)

↓

(4,3)

↓

(4,4)

Total Path Cost

Number of moves

8

Cost

$$8 \times 1 = 8$$

Search Diagram

S

↓

↓

↓

↓

→

↓

→

→

G

Result

BFS guarantees the shortest path.

Optimal Cost

8

Code:

```
from collections import deque
```

```
grid = [  
    ['S',0,0,0,0],  
    [1,1,0,1,0],  
    [0,0,0,1,0],  
    [0,1,0,0,0],  
    [0,1,1,1,'G']  
]
```

```
rows = len(grid)  
cols = len(grid[0])
```

```
start = (0,0)  
goal = (4,4)
```

```

moves = [(-1,0),(1,0),(0,-1),(0,1)]

queue = deque()
queue.append((start,[start]))

visited = set()

while queue:

    current,path = queue.popleft()

    print("\nNode Expanded:",current)
    print("Queue:",list(queue))

    if current==goal:
        print("\nGoal Reached")
        print("Optimal Path")
        print(path)
        print("Total Cost =",len(path)-1)
        break

    if current in visited:
        continue

    visited.add(current)

    for dx,dy in moves:

        x=current[0]+dx
        y=current[1]+dy

        if 0<=x<rows and 0<=y<cols:
            if grid[x][y]!=1 and (x,y) not in visited:
                queue.append(((x,y),path+[(x,y)]))

outcome:

```

```

Node Expanded: (0, 0)
Queue: []

Node Expanded: (0, 1)
Queue: []

Node Expanded: (0, 2)
Queue: []

Node Expanded: (1, 2)
Queue: [(0, 3), [(0, 0), (0, 1), (0, 2), (0, 3)]]

Node Expanded: (0, 3)
Queue: [(2, 2), [(0, 0), (0, 1), (0, 2), (1, 2), (2, 2)]]

Node Expanded: (2, 2)
Queue: [(0, 4), [(0, 0), (0, 1), (0, 2), (0, 3), (0, 4)]]

Node Expanded: (0, 4)
Queue: [(3, 2), [(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (3, 2)], (2, 1), [(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (2, 1)]]

Node Expanded: (3, 2)
Queue: [(2, 1), [(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (2, 1)], (1, 4), [(0, 0), (0, 1), (0, 2), (0, 3), (0, 4), (1, 4)]]

Node Expanded: (2, 1)
Queue: [(1, 4), [(0, 0), (0, 1), (0, 2), (0, 3), (0, 4), (1, 4)], (3, 3), [(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (3, 2), (3, 3)]]

Node Expanded: (1, 4)
Queue: [(3, 3), [(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (3, 2), (3, 3)], (2, 0), [(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (2, 1), (2, 0)]]

Node Expanded: (3, 3)
Queue: [(2, 0), [(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (2, 1), (2, 0)], (2, 4), [(0, 0), (0, 1), (0, 2), (0, 3), (0, 4), (1, 4), (2, 4)]]

Node Expanded: (2, 0)
Queue: [(2, 4), [(0, 0), (0, 1), (0, 2), (0, 3), (0, 4), (1, 4), (2, 4)], (3, 4), [(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (3, 2), (3, 3), (3, 4)]]

Node Expanded: (2, 4)
Queue: [(3, 4), [(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (3, 2), (3, 3), (3, 4)], (3, 0), [(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (2, 1), (2, 0), (3, 0)]]

Node Expanded: (3, 4)
Queue: [(3, 0), [(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (2, 1), (2, 0), (3, 0)], (3, 4), [(0, 0), (0, 1), (0, 2), (0, 3), (0, 4), (1, 4), (2, 4), (3, 4)]]

Node Expanded: (3, 0)
Queue: [(3, 4), [(0, 0), (0, 1), (0, 2), (0, 3), (0, 4), (1, 4), (2, 4), (3, 4)], (4, 4), [(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (3, 2), (3, 3), (3, 4), (4, 4)]]

Node Expanded: (3, 4)
Queue: [(4, 4), [(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (3, 2), (3, 3), (3, 4), (4, 4)], (4, 0), [(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (2, 1), (2, 0), (3, 0), (4, 0)]]

Node Expanded: (4, 4)
Queue: [(4, 0), [(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (2, 1), (2, 0), (3, 0), (4, 0)]]

Goal Reached
Optimal Path
[(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (3, 2), (3, 3), (3, 4), (4, 4)]
Total Cost = 8

```

Question 3 : Autonomous Rescue Robot

Problem Statement

A rescue robot must reach survivors inside a disaster building.

Some rooms are blocked.

Risky zones increase travel cost.

Robot only knows adjacent cells.

A) Constraint Analysis

Movement

Robot moves only

- Up
- Down
- Left
- Right

This reduces possible actions.

Obstacles

Blocked rooms cannot be entered.

Robot searches alternative paths.

Limited Sensors

Robot sees only neighbouring rooms.

Knowledge is incomplete.

Risk Zones

Normal Move

Cost = 1

Risk Zone

Cost = 3

Robot prefers safer paths.

Dynamic Knowledge

Robot updates its internal map whenever new rooms become visible.

B) Online Search using Uniform Cost Search

Step 1

Start at

S

Cost = 0

Step 2

Observe adjacent rooms.

Blocked rooms are removed.

Step 3

Insert valid neighbours into priority queue.

Priority Queue

[(A,1),(B,3)]

Step 4

Expand lowest-cost node first.

Choose

A

Cost = 1

Step 5

Repeat until Goal is reached.

Uniform Cost Search Table

Expanded Cost

S	0
A	1
C	2
D	3
G	5

Total Cost

5

C) AI Concepts

Intelligent Agent

The rescue robot

- senses environment
- updates knowledge
- plans actions
- reaches survivor

Rationality

A rational agent

- minimizes cost
- avoids dangerous areas
- safely reaches survivor

Environment Classification

Property	Type
Observability	Partially Observable
Agent	Single Agent
Environment	Dynamic
Nature	Stochastic
Decision	Sequential
State Space	Discrete

D) Justification

Uniform Cost Search is suitable because

- Handles different movement costs.
- Always expands the least-cost path first.
- Finds the optimal path even with risky zones.

- Works well in dynamic environments when combined with online updates.
- Ensures safe and cost-efficient rescue.

Flowchart

START

|

Sense Adjacent Cells

|

Update Internal Map

|

Generate Valid Moves

|

Calculate Cost

|

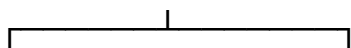
Insert into Priority Queue

|

Expand Lowest Cost Node

|

Goal Reached?



No

Yes

|

|

Repeat

Rescue Survivor

|

STOP

Code:

```
import heapq

grid = [
['S',0,0,0,0],
[0,'R',1,'R',0],
[0,0,0,1,0],
[1,'R',0,0,0],
[0,0,0,'R','G']
]

rows = len(grid)
cols = len(grid[0])

start = (0,0)
goal = (4,4)

moves = [(-1,0),(1,0),(0,-1),(0,1)]

pq = []

heapq.heappush(pq,(0,start,[start]))

visited = {}

while pq:

    cost,node,path = heapq.heappop(pq)

    print("Visited:",node," Cost =",cost)

    if node==goal:
        print("\nGoal Reached")
        print("Optimal Path")
        print(path)
        print("Minimum Cost =",cost)
        break

    if node in visited and visited[node]<=cost:
        continue

    visited[node]=cost

    for dx,dy in moves:

        x=node[0]+dx
        y=node[1]+dy
```

```
if 0<=x<rows and 0<=y<cols:
```

```
    if grid[x][y]!=1:
```

```
        extra=1
```

```
        if grid[x][y]=='R':
```

```
            extra=3
```

```
        heapq.heappush(
```

```
            pq,
```

```
            (
```

```
                cost+extra,
```

```
                (x,y),
```

```
                path+[(x,y)]
```

```
            )
```

```
        )
```

Output:

```

Visited: (0, 0) Cost = 0
Visited: (0, 1) Cost = 1
Visited: (1, 0) Cost = 1
Visited: (0, 0) Cost = 2
Visited: (0, 0) Cost = 2
Visited: (0, 2) Cost = 2
Visited: (2, 0) Cost = 2
Visited: (0, 1) Cost = 3
Visited: (0, 3) Cost = 3
Visited: (1, 0) Cost = 3
Visited: (2, 1) Cost = 3
Visited: (0, 2) Cost = 4
Visited: (0, 4) Cost = 4
Visited: (1, 1) Cost = 4
Visited: (1, 1) Cost = 4
Visited: (2, 0) Cost = 4
Visited: (2, 2) Cost = 4
Visited: (0, 1) Cost = 5
Visited: (0, 3) Cost = 5
Visited: (1, 0) Cost = 5
Visited: (1, 4) Cost = 5
Visited: (2, 1) Cost = 5
Visited: (2, 1) Cost = 5
Visited: (3, 2) Cost = 5
Visited: (0, 4) Cost = 6
Visited: (1, 1) Cost = 6
Visited: (1, 3) Cost = 6
Visited: (2, 2) Cost = 6
Visited: (2, 4) Cost = 6
Visited: (3, 1) Cost = 6
Visited: (3, 3) Cost = 6
Visited: (4, 2) Cost = 6
Visited: (0, 3) Cost = 7
Visited: (1, 4) Cost = 7
Visited: (1, 4) Cost = 7
Visited: (2, 1) Cost = 7
Visited: (3, 2) Cost = 7
Visited: (3, 2) Cost = 7
Visited: (3, 2) Cost = 7
Visited: (3, 4) Cost = 7
Visited: (3, 4) Cost = 7
Visited: (4, 1) Cost = 7
Visited: (4, 1) Cost = 7
Visited: (1, 3) Cost = 8
Visited: (2, 4) Cost = 8
Visited: (3, 1) Cost = 8
Visited: (3, 3) Cost = 8
Visited: (4, 0) Cost = 8
Visited: (4, 2) Cost = 8
Visited: (4, 4) Cost = 8

Goal Reached
Optimal Path
[(0, 0), (0, 1), (0, 2), (0, 3), (0, 4), (1, 4), (2, 4), (3, 4), (4, 4)]
Minimum Cost = 8

```

Conclusion

The rescue robot behaves as an **intelligent, rational, goal-based agent** operating in a **partially observable and dynamic environment**. By combining **online sensing** with **Uniform Cost Search**, it continually updates its knowledge, avoids blocked or risky areas, and reaches the survivor with the **minimum possible total cost**, making it an effective AI solution for disaster response.